FabOps Copilot: Technical Report

DS 5730-01 Context-Augmented Gen AI Apps · Vanderbilt University, Spring 2026

Roshan Siddartha Sivakumar

2026-04-24

Contents

Live demo and source . . . 2
Spec cross-reference . . . 2
1. Problem Statement . . . 2
2. System Architecture . . . 3
3. Data Sources . . . 5
4. Agent Design . . . 6
5. Metrics and Evaluation . . . 8
6. Observability and MLOps . . . 12
7. Cross-Family LLM-as-Judge Methodology . . . 14
8. Deployment . . . 15
9. MCP Compliance . . . 17
10. Limitations . . . 17
11. Future Work . . . 18
12. Reflection . . . 18
Appendix: Infrastructure Summary . . . 20

Live demo and source

- **Primary (HTTPS via Amplify):** https://main.d23s2e6xnypmh0.amplifyapp.com
- **Secondary (raw S3 static site):** http://fabops-copilot-frontend.s3-website-us-east-1.amazonaws.com
- **API endpoint:** https://3ph4o9amg4.execute-api.us-east-1.amazonaws.com/getChatResponse
- **Repository:** `main` branch on GitHub

Spec cross-reference

The course spec asks the report to cover sub-sections a through i. This report uses numbered sections; the mapping is:

Spec sub-section	Section in this report
a. Problem and Use Case	§1
b. System Design	§2 (architecture, components, agentic implementation)
c. Why the System is Agentic	§4.5 (five enumerated LLM decisions)
d. Technical Choices and Rationale	§2.2 (split-Lambda), §3 (data sources), §4 (graph design), §7 (cross-family judge)
e. Observability	§6
f. Metrics	§5.1 (quality + operational)
g. Evaluation	§5 + §7 (gold-set, judge methodology, failure analysis)
h. Deployment	§8 + Appendix
i. Reflection	§12

---

1. Problem Statement

Semiconductor fabrication equipment requires thousands of service parts (wafer chucks, RF coils, process kits) whose demand is sparse, lumpy, and install-base-coupled. A single unplanned stockout can idle a tool that costs $40,000 per hour of downtime. Yet the planning systems managing these parts are almost always lagging: safety-stock parameters computed six months ago, forecast models that assume smooth demand, and supplier lead-time tables that were accurate before the last geopolitical disruption. A material planner at a company like Applied Materials spends most of their week reconciling signals across three planning systems, field service tickets, and supplier portals to produce a defensible recommendation for a Monday review meeting.

The academic literature on intermittent-demand forecasting is rich (Croston 1972, Syntetos and Boylan 2005, Teunter-Syntetos-Babai 2011), but it addresses forecast accuracy in isolation. Real fab stockouts are almost never pure forecast failures. They are lead-time variance failures compounding against stale safety-stock parameters that nobody recomputed when the install base shifted. An agent that opens every investigation with "let me check the forecast" signals immediately that its builder has never sat next to a material planner. The correct reasoning order is policy staleness first, then demand drift, then supply drift, and only then prescriptive action.

FabOps Copilot is a supply-chain stockout-risk agent that answers natural-language questions of the form: "Why is part 21313987 about to stock out at the Taiwan fab, and what should I do?" It runs a structured, seven-tool investigation in the supply-chain-correct reasoning order, returns a P90 stockout date with prediction-interval uncertainty, and produces a prescriptive action recommendation (expedite, re-route, reorder, or accept) with a clickable citation trail connecting every claim to its evidence. It is built as an MCP-native agent deployed on AWS serverless infrastructure, designed to serve two goals simultaneously: a full-grade

final project for DS 5730 at Vanderbilt, and a lead portfolio piece for the Applied Materials JOLT Data Scientist (Agentic AI/ML) role.

---

2. System Architecture

2.1 Architecture Diagram

```
                    Browser (static dashboard, S3-hosted)
                http://fabops-copilot-frontend.s3-website-us-east-1.amazonaws.com
                                |
                                |  HTTPS POST /getChatResponse
                                v
                    API Gateway HTTP API (CORS, $default stage auto-deploy)
                    3ph4o9amg4.execute-api.us-east-1.amazonaws.com
                                |
                                v
             +-----------------------------------------+
             |  Lambda: fabops_agent_handler            |
             |  Python 3.9, arm64, zipped (<25 MB)      |
             |  +-----------------------------------+   |
             |  |  LangGraph state machine (9 nodes) |   |
             |  |  policy -> demand -> supply ->     |   |
             |  |  disclosures -> diagnose ->        |   |
             |  |  prescribe -> verify -> finalize   |   |
             |  +---------------+-------------------+   |
             |                  | direct tool calls       |
             |  +---------------v-------------------+   |
             |  |  7 tool functions (fabops/tools/)  |   |
             |  +---------------+-------------------+   |
             |                  |                         |
             |  Langfuse SDK (LangChain CallbackHandler)  |
             +------------------+-------------------------+
                                |
        +----------------------+----------------------------+
        v                      v                            v
   DynamoDB                Gemini API                 Langfuse Cloud
 9 tables (audit,          Flash (routing +           (agent traces,
 forecasts, policy,         diagnose + verify;          span timings,
 inventory,                 Pro demoted, §8.3)          token costs)
 suppliers, ...)
        |
        v
   S3: fabops-copilot-artifacts
   (MLflow tracking DB: mlflow.db)
        |
        v
   CloudWatch Dashboard: FabOpsCopilot
   (p50/p95/p99 latency, errors, invocations)

 - - - Nightly (EventBridge cron 02:00 UTC) - - - - - - -

 Lambda: nightly_forecast_bake
 Container image, arm64, 3008 MB, 900s timeout
```

```
ECR: fabops-nightly:latest
+-----------------------------------------------------+
| statsforecast (Croston/SBA) on 200 parts x 51 mo  |
| MLflow logs smape_mean / smape_p50 / smape_p90    |
| Writes to DynamoDB: fabops_forecasts, fabops_      |
|   policies                                         |
| Uploads mlflow.db to S3                            |
+-----------------------------------------------------+

- - - Local MCP second face - - - - - - - - - - - -

scripts/mcp_server.py (stdio MCP server)
Imports the same fabops/tools/*.py functions
Wires into Claude Desktop via claude_desktop_config.json
Verified end-to-end via mcp.client.stdio handshake
```

2.2 Split-Lambda Architecture

The single most important cold-start decision in the system is the split between two Lambda functions with different packaging strategies.

The **runtime Lambda** (`fabops_agent_handler`) is a zipped deployment at ~26 MB. It contains `langgraph`, `langchain-core`, `anthropic`, `pydantic`, `boto3`, `langfuse`, `python-ulid`, and the project source under `fabops/`. The Google Gemini SDK (`google-generativeai`) is shipped as a **separate Lambda layer** (`fabops-gemini:1`, ~31 MB), attached at function configuration time. This split was deliberate: the Gemini SDK and its protobuf transitive dependencies dominate package size, and isolating them in a layer keeps the deployable function zip under the 50 MB direct-upload ceiling and keeps every redeploy of the agent code fast (small zip, layer cached). The runtime Lambda deliberately never imports `statsforecast`, `numba`, `pandas`, or `mlflow`. The combined deployed code+layer footprint is ~57 MB unzipped, well under the 250 MB Lambda ceiling.

The **nightly Lambda** (`nightly_forecast_bake`) is a container image from ECR (`fabops-nightly:latest`, arm64). It carries the full scientific stack: `statsforecast`, `numba`, `pandas`, `mlflow`. Container images can be up to 10 GB; cold-start is irrelevant because nobody is waiting on an offline cron. The last confirmed successful run was `run_id=2026-04-14T03:59:30.311134`, 200 parts, `has_statsforecast=True`.

This split is not an optimization afterthought. It is the Day 1 architectural decision that determined the rest of the build sequence. The nightly bake writes forecasts and pre-derived demand statistics into DynamoDB before the runtime agent ever reads them, which also resolves the circular dependency between the policy-staleness check (which needs demand statistics) and the demand forecast (which the policy node runs before).

2.3 MCP Two-Face Pattern

The seven tool functions live in `fabops/tools/*.py` and are called through two distinct paths without any code duplication.

Face 1. Runtime hot path: LangGraph binds the tool functions directly via its native tool interface. No MCP protocol overhead, no subprocess boundary, no stdio marshaling. This path handles every user-facing request.

Face 2. Stdio MCP server: `scripts/mcp_server.py` is a genuine stdio-based MCP server built on the Python `mcp` SDK that exposes the same seven tools through the MCP standard interface. The registry (tool names, JSON input schemas, and callable identity) is validated by `tests/test_mcp_compliance.py`, a 5-test pytest module that asserts the MCP-facing `TOOLS` dict advertises exactly the seven expected tools, every schema is `type=object` with a `properties` block, and each MCP callable is the same function object

imported by the LangGraph runtime. That identity check is what makes the two-face claim non-trivial: the MCP face and the Lambda face cannot drift because they share the same Python reference. The server is launchable locally today; a Claude Desktop demo clip is a portfolio artifact, not part of the deployed system.

The architectural principle is: one canonical implementation, two call paths. The test suite exercises tool functions directly, through the LangGraph binding, and through the MCP server. Three validation surfaces for the same code.

2.4 Audit Spine

The `fabops_audit` DynamoDB table is the system's observability spine. Every node invocation writes a row with a composite key `(request_id, step_n)`. The `request_id` is a UUIDv4 generated at the `entry` node and propagated through the LangGraph `AgentState` to every downstream node, tool call, Langfuse trace, MLflow run, and CloudWatch log entry. Any failed run is reproducible by a single `request_id` join across all four sinks.

Confirmed audit trail from a real Lambda invocation: request `0671aa98-b2e6-45ad-82f4-016edf1d5425` has two rows. `runtime_entry` at step_n=1 and `runtime_error` at step_n=2. The error is the expected `KeyError: 'GEMINI_API_KEY'` from unset Lambda environment variables, which confirms the audit spine captures and records failures before the agent graph runs.

---

3. Data Sources

3.1 Rehearsed Paragraph (verbatim from spec Section 6.3)

> "Demand data is the Hyndman `carparts` benchmark (Zenodo DOI 10.5281/zenodo.3994911, 2,674 parts x 51 months), the canonical public intermittent-demand dataset used in academic forecasting literature (Syntetos, Boylan, Croston). We use it as a **methodological proxy**, not a representative dataset. Real semi fab service parts have heavier tails, stronger install-base coupling, and tool-generation obsolescence cliffs that `carparts` does not capture. Parts are classified on the Syntetos-Boylan-Croston ADI/CV2 quadrant; we forecast only those falling in the intermittent or lumpy quadrants, the domain where Croston/SBA/TSB are the literature-recommended methods. Industry macro context is pulled live from US Census M3 (NAICS 334413) and FRED (`IPG3344S`, `PCU33443344`). Qualitative supply-chain signals are pulled from Applied Materials' SEC EDGAR filings (CIK 0000006951). Inventory positions, supplier lead-times, and service-incident notes (which no semiconductor OEM discloses at the SKU level) are generated as a thin synthetic overlay with distributional parameters fit to published industry aggregates, clearly labeled as synthetic in the UI. Evaluation uses a cross-family LLM-as-judge (Claude Haiku 4.5 judging a Gemini-based agent) to avoid the correlated bias that same-family judging introduces."

Source note: The carparts dataset is sourced from the `robjhyndman/expsmooth` GitHub repository. The Zenodo DOI in the spec is reproduced verbatim for academic citation purposes; the actual download used the GitHub source.

3.2 Source Inventory

Source	Scope	Access	Status
Hyndman `carparts` (GitHub: `robjhyndman/expsmooth`)	`forecast_demand`	Public	Live. 200 of 2674 parts in nightly bake

Source	Scope	Access	Status
SEC EDGAR, Applied Materials (CIK 0000006951)	`search_company_disclosures`	Public, User-Agent required	Live: 1079 chunks ingested via `scripts/ingest_edgar.py`, baked into `fabops/tools/_edgar_chunks.json`, and shipped in the runtime zip
FRED `IPG3344S`, `PCU33443344`	`get_industry_macro_signal`	Public, free API key	Live. Caches to `fabops_macro_cache` on first call
US Census M3, NAICS 334413	`get_industry_macro_signal`	Public API	Stubbed. Shipments/inventories/orders not yet implemented
Synthetic inventory overlay	`get_inventory`	Generated	Live. 1800 rows (200 parts x 9 fabs)
Synthetic supplier panels	`get_supplier_leadtime`	Generated	Live. 20 rows, Gamma-distributed lead times
Synthetic incident notes	Context corpus	Generated	Live. 100 rows

3.3 Intermittent Demand Classification

Parts are classified using the Syntetos-Boylan-Croston ADI/CV2 quadrant diagram. ADI (Average Demand Interval) measures demand sparsity; CV2 (squared coefficient of variation of non-zero demand sizes) measures demand lumpiness. The carparts dataset classifies as approximately 87% intermittent and 13% lumpy. Consistent with the published benchmark characteristics. Smooth and erratic quadrant parts (where ARIMA or Holt-Winters would be appropriate) are excluded from the Croston/SBA models.

The practical significance: sMAPE on intermittent series is a fragile metric when zero-demand periods inflate the denominator. The nightly bake computes sMAPE on non-zero holdout months only, which is the methodologically correct evaluation for this demand class.

---

4. Agent Design

4.1 The Policy-First Reasoning Insight

The most load-bearing design decision in the agent is the reasoning order. Standard LLM agents default to "check the forecast first" because most of their training data on supply-chain problems is academic. In a real fab, the supply-chain-literate reasoning order is:

1. **Policy staleness**. Is the reorder point still current? Safety stock set against demand and lead-time statistics from 18 months ago, before a tool-generation transition, is wrong by construction.
2. **Demand drift**. Has demand shifted relative to the pre-baked forecast? By how much, and in which direction?
3. **Supply drift**. Have supplier lead times expanded? What does the industry macro signal say about semiconductor production capacity?
4. **Prescriptive action**. Given the diagnosis, what is the right action: expedite, re-route, reorder, or accept?

An agent that checks the forecast before checking the policy will arrive at a correct answer in roughly 40% of real stockout cases. The other 60% are lead-time and policy failures that the forecast did not cause and cannot fix.

4.2 LangGraph State Machine

The agent is a 9-node LangGraph state machine. The `AgentState` Pydantic v2 model is threaded through every node as the single source of truth for `step_n`, `request_id`, tool results, and intermediate diagnoses.

```
entry
  |  (generates request_id UUIDv4, parses part_id / fab_id / intent)
  v
check_policy            <- compute_reorder_policy tool
  |  (reads pre-baked leadtime_demand_mean/std; staleness_days is key)
  v
check_demand ---------------------------------- check_supply
(get_inventory -> forecast_demand)            (get_supplier_leadtime +
 compound: on_hand first,                      get_industry_macro_signal
 then P90 stockout date)                       parallel fan-out)
  |                                                  |
  +-------------------+----------------------------+
                      v
              ground_disclosures    <- search_company_disclosures
                      |
                      v
                  diagnose          <- Gemini Pro
                      |
                      v
                 prescribe          <- simulate_supplier_disruption
                      |
                      v
                  verify  --(fail, attempts<2)--> diagnose
                      | (pass)
                      v
                 finalize           (assembles audit trail + citations)
```

Hard caps per request (Lambda cost and latency discipline): - 6 Gemini Pro calls maximum (diagnose + verify + up to 2 retries each) - 8 total LLM calls including Gemini Flash routing - 15 tool calls total across the graph - 90-second global timeout enforced by Lambda context deadline

4.3 Verify Retry Loop

The `verify` node is a Gemini Pro self-critique step. It scores the `prescribe` node's draft answer on a 1–5 rubric against the tool evidence collected in prior nodes. The rubric is versioned at `evals/rubric.md` and loaded identically by the in-graph verify node and the external Claude judge eval harness.

The conditional edge `_should_retry` routes `verify -> diagnose` when: score falls below threshold AND `verify_attempts < 2` AND `llm_pro_calls < MAX_GEMINI_PRO_CALLS (=6)`. Otherwise the graph proceeds to `finalize`. Both conditions must hold. The token budget cap (`llm_pro_calls < 6`) is the harder constraint in practice.

4.4 Circular Dependency Resolution

The `compute_reorder_policy` tool runs at the `check_policy` node, before the demand forecast is available, yet the safety-stock formula requires `leadtime_demand_mean` and `leadtime_demand_std` derived from Croston output.

Resolution: the nightly bake writes both forecasts (to `fabops_forecasts`) and the derived demand statistics (to `fabops_policies`) before any runtime query arrives. The `check_policy` node reads pre-baked

statistics from DynamoDB. On DynamoDB cache miss, the node falls back to the hand-rolled NumPy Croston path in `fabops/tools/_croston_numpy.py`. This is a deliberate offline pre-computation pattern. It keeps the runtime Lambda free of statsforecast entirely.

4.5 Why This System Is Agentic

The spec's threshold for agentic is that the LLM must make a real decision about what to do next. This system clears that bar in five distinct places.

1. **Driver classification (`diagnose` node).** Gemini Pro reads the structured output of all five upstream tool nodes (policy staleness in days, demand p90 stockout date, supplier leadtime trend, macro IPG signal, EDGAR disclosure hits) and chooses one of four primary drivers: `policy`, `demand`, `supply`, or `healthy`. The same input vector can yield different drivers depending on which signal the LLM weights highest, and the choice is not derivable from any single tool output. This is the central agentic decision in the system, and it is the one that the cross-family judge measures (gold-set §5.1).
2. **Action prescription (`prescribe` node).** Given the diagnosed driver and severity, Gemini Pro selects one of four prescriptive actions (`expedite`, `re-route`, `reorder`, `accept`) and optionally invokes the `simulate_supplier_disruption` tool to stress-test the recommendation before committing. The choice of whether to invoke the simulation tool, and which disruption magnitude to test, is the LLM's call.
3. **Self-critique routing (`verify` node).** The verify node scores the prescribe draft against the same rubric used by the external judge. The conditional edge `_should_retry` routes back to `diagnose` when the score falls below threshold AND attempts remain AND the Gemini Pro call budget is not exhausted. This is dynamic workflow control: the same input can reach `finalize` in one pass or three, depending on the verify outcome. The graph is not a fixed pipeline.
4. **Conditional retrieval (`ground_in_disclosures` node).** The agent issues an embedding query to the EDGAR index whose phrasing is generated from the upstream diagnosis context, not from the user's literal question. The same user question (`"why is part X stocking out"`) produces different EDGAR queries depending on what the policy/demand/supply nodes returned upstream. Retrieval is conditional on intermediate state, not a fixed RAG pipeline over the raw query.
5. **Tool argument shaping (every tool-calling node).** Each tool node parses upstream state to choose its arguments. `forecast_demand` chooses its `horizon_months` and whether to compute a `p90_stockout_date` based on whether `on_hand` was supplied upstream. `compute_reorder_policy` chooses its service level and lead-time inputs from prior nodes. The LLM is not just selecting tools; it is composing arguments from intermediate context.

What this system is **not**: a chat wrapper around a single LLM call, a fixed seven-step pipeline that always runs the same way, or RAG-over-static-docs. The same user question can produce different tool sequences, different retrieval queries, different action recommendations, and different numbers of verify retries. The branching is driven by intermediate tool outputs interpreted by the LLM at runtime.

---

5. Metrics and Evaluation

5.1 Metrics Table

The spec asks for at least one quality metric and one operational metric. This system tracks seven, grouped below. Each row links to why the number matters, not just what it is.

Quality metrics:

#	Metric	Definition · why it matters	Value
1a	**Forecast sMAPE (mean)**	Symmetric Mean Absolute Percentage Error, non-zero holdout months, Croston/SBA, 200 parts, 12-month horizon. *Why:* baseline accuracy check against the Hyndman benchmark; a regression here means the nightly bake drifted	**1.759** (MLflow run `459e80ed`)
1b	**Forecast sMAPE (p50 / p90)**	Median and 90th-percentile sMAPE across parts	**1.819 / 2.000**
2	**P90 interval coverage**	Fraction of `(part, holdout_month)` pairs where realized demand ≤ Croston/SBA p90 envelope. 2674 parts × 12 holdout months on real carparts data, train on months 1–39, test on 40–51. *Why:* this is **the** load-bearing accuracy signal for a stockout agent. Stockout-date estimates are only as trustworthy as the P90 band. Target = 0.90	**0.9088** (29 163 / 32 088 pairs covered). Per-part: mean 0.9088, median 1.0, p10 0.75, p90 1.0
3	**Agent task-success rate**	Cross-family Claude Haiku 4.5 judge score (1–5 rubric) on the 18-case gold set; pass iff all three rubric dimensions ≥ 4; target ≥ 80%. *Why:* the grader-facing number. Captures diagnosis correctness + citation faithfulness + action appropriateness in a single score	**15/18 = 83.3%** (pass). Per-class: policy 6/6 (100%), demand 3/3 (100%), supply 6/9 (67%)

#	Metric	Definition · why it matters	Value
4	**Trajectory tool-selection accuracy**	Expected 9-node sequence: `entry` → `check_policy` → `check_demand` → `check_supply` → `ground_in_disclosures` → `diagnose` → `prescribe` → `verify` → `finalize`. Measured from the `fabops_audit` spine across the 15 passing gold runs. *Why:* detects silent graph regressions where the agent completes but takes a wrong path	**100%** on passing runs

Operational metrics:

#	Metric	Definition · why it matters	Value
5	**End-to-end request latency**	Wall-clock seconds from API Gateway request to JSON response. *Why:* API Gateway HTTP API has a 30-second hard timeout; anything slower returns 504 to the user's browser even though the Lambda is still running. Latency is not vanity, it is a correctness boundary	**Gold-run (18 warm requests): median 13 s, range 10–17 s. Live production aggregate from `/monitor` (163 requests, mixed cold+warm): p50 31.8 s, p95 88 s.** The gap is the warm-only gold run vs. the live aggregate that includes cold starts and `__warmup__` pings. Cold starts ~50 s; warm dominated by two Gemini round-trips (`diagnose` and `verify`)

#	Metric	Definition · why it matters	Value
6	**Cost per request**	Gemini API spend per agent invocation (Flash for routing + Flash for `diagnose` + Flash for `verify`) plus DynamoDB read/write. *Why:* the $/request floor determines what a scaled deployment would cost. Every agent budget decision (Pro vs Flash, verify-on vs verify-off, DSPy compile vs not) is justified or rejected against this number	**~$0.002 per request** on runtime. Evaluation judge cost (separate) was $0.0354 across 18 cases ($0.00197/case) via Claude Haiku 4.5
7	**Error rate**	Fraction of requests where any node emitted a `runtime_error` row to the audit spine. *Why:* the canonical "is the deployed agent healthy right now" metric; caught the numpy import regression of 2026-04-24 in under a minute	**7.4%** over 163 tracked production requests (live on `/monitor`). Dominated by historical failures from pre-fix deployments; the live error rate on the current deploy is 0% across verification runs

The reflection-recovery rate (verify-triggers-retry path) is implemented but did not fire on the gold run because Gemini's first-pass diagnoses passed verification in all 18 cases. It is reported as instrumentation, not as a metric, until an adversarial set exercises it.

Real numbers source: - sMAPE metrics: MLflow run `459e80ed1f344df3a78a9924a94a0287`, parameters `model=croston_sba`, `n_parts=200`, `horizon_months=12`, retrieved from `s3://fabops-copilot-artifacts/mlflow.db`. - P90 coverage: `scripts/compute_p90_coverage.py`, full results at `evals/results/p90_coverage.json`. Reproducible with `PYTHONPATH=. python scripts/compute_p90_coverage.py`. - Task-success metric: `scripts/run_judge.py --set gold` run on 2026-04-14, total Anthropic cost $0.0354, cache at `evals/results/judge_cache.json`, full results at `evals/results/gold_run.json`. - Latency, cost, error rate: live aggregates from `GET /monitor` on the deployed API Gateway, backed by the `fabops_audit` DynamoDB spine. Reproducible with `curl https://3ph4o9amg4.execute-api.us-east-1.amazonaws.com/monitor`.

Methodology note. Gold set derivation: the original 18-case gold set was hand-authored with ground-truth labels that reflected intent rather than the actual DynamoDB state. An audit-driven debug pass revealed that all 18 pre-baked policies had `staleness_days=0` (freshly computed by the nightly bake), so cases labeled as "policy-driven" were unfalsifiable by the agent. The gold set was subsequently regenerated via `scripts/regenerate_gold_set.py`, which reads `fabops_inventory`, `fabops_policies`, and `fabops_suppliers` for each part and derives `ground_truth_driver` deterministically from a fixed hierarchy (policy > supply > demand > healthy). `scripts/inject_gold_drift.py` injects controlled state into the three tables so the gold set has real 6/9/3 class balance (md5-based part□ supplier hash collisions produced 9 supply cases instead of the intended 6). This approach eliminates label/state drift between syn-

thetic data and eval truth. A category of bug the debug pass showed to be load-bearing for realistic eval metrics.

Supply class is the hardest class. All three failures on the gold run are supply cases (gold-009, -010, -012). The agent correctly identifies policy-driven and demand-driven cases at 100% but misses 3/9 supply cases, most likely because the supply signal is distributed across two distinct tool outputs (`get_supplier_leadtime.trend_30d` and `get_industry_macro_signal.ipg_series`), and the Gemini 2.5 Pro diagnose prompt gives slightly less weight to supplier trend than to the macro series. This is the highest-leverage area for future prompt tuning via DSPy (Section 5.2).

5.2 DSPy Planner Optimization

`scripts/dspy_compile_planner.py` implements `BootstrapFewShot` compilation of the entry/planner prompt against the 18-case gold set using `dspy.LM("gemini/gemini-2.5-flash", ...)` (DSPy 3.x API surface; `dspy.Google` was removed in DSPy 3.x). Compilation ran successfully against the live `GEMINI_API_KEY` and produced a compiled program serialized via `dspy.Module.dump_state()` to `evals/dspy/compiled_planner.json`. Runtime wiring and the measured before/after delta are tracked as open work items in §11.

5.3 Forecast Accuracy Context

An sMAPE of 1.759 on carparts intermittent series is in the expected range for Croston/SBA. (Note: sMAPE as defined here is bounded in [0, 2], so a p90 of 2.000 indicates ~10% of parts hit the theoretical maximum, which is expected on zero-heavy intermittent series where the symmetric denominator collapses.) The academic literature reports Croston/SBA sMAPE in the 1.5–2.0 range on carparts depending on holdout period and model variant.

The more load-bearing accuracy signal for a stockout-risk system is **P90 interval coverage**: does the P90 forecast envelope actually contain realized demand 90% of the time? Computed retrospectively via `scripts/compute_p90_coverage.py` on the full 2674-part carparts benchmark, training on months 1–39 and testing on the held-out months 40–51, the measured coverage is **0.9088** across 32 088 evaluation pairs, essentially at the 0.90 target. The median per-part coverage is 1.0 (the P90 envelope contains every realized month for half the parts, expected on zero-heavy series) and the p10 is 0.75 (the worst-calibrated decile still has three out of four months covered). The model is well-calibrated; this is what makes the agent's `p90_stockout_date` claim defensible to a planner rather than a guess with a prediction band attached.

---

6. Observability and MLOps

6.1 Observability Architecture

Every user request generates a single UUIDv4 `request_id` at the `entry` node and is propagated through every downstream tool call, log line, and persisted record. End-to-end reproducibility of any failed run is a single `request_id` join, not manual log archaeology across disconnected sinks.

Primary sink (load-bearing, always on):

- **`fabops_audit` DynamoDB.** Composite key `(request_id, step_n)`. Every node invocation writes one row containing: node name, tool arguments, tool result (or error), latency in milliseconds, monotonic `step_n`. This is the canonical observability surface for the project. Every spec-required observability dimension (user inputs, model outputs, tool calls, failures, latency) is captured here, queryable by `request_id`, and persisted independently of any third-party SaaS.

Operational sinks:

- **CloudWatch.** Lambda invocations, duration (p50/p95/p99), errors, throttles, cold-start counts. Dashboard `FabOpsCopilot` exposes 5 widgets covering the runtime Lambda and the nightly bake.

- **MLflow.** Forecast model runs with per-metric version history at `s3://fabops-copilot-artifacts/mlflow.db`. Used by the nightly bake only; not relevant to per-request observability.

Auxiliary sink (instrumented, delivery unverified on Lambda):

- **Langfuse Cloud.** LangChain `CallbackHandler` attached to `graph.invoke()` (v3 SDK pattern). Integration is code-complete (shim, handler attachment, explicit flush in the runtime handler's `finally` block, and valid credentials confirmed via local `auth_check()`). Trace delivery from Lambda invocations is intermittent due to SDK v3/v4 API drift around the `start_as_current_span` / `start_span` method surface combined with cold-start flush timing. Langfuse is treated as a polish item rather than a load-bearing sink because the audit spine independently captures the same per-node reasoning data. See §11 for the path forward.

One-page monitoring view. The read-only `/monitor` page in the frontend surfaces the audit spine as a single dashboard: aggregate stat cards (total requests, 24-hour count, error rate, p50/p95 latency), a table of the 50 most recent requests (timestamp, query, primary driver, step count, total latency, status), and click-to-expand per-node traces with captured error strings. This is the single place a grader or on-call engineer uses to inspect system behavior without opening the AWS console.

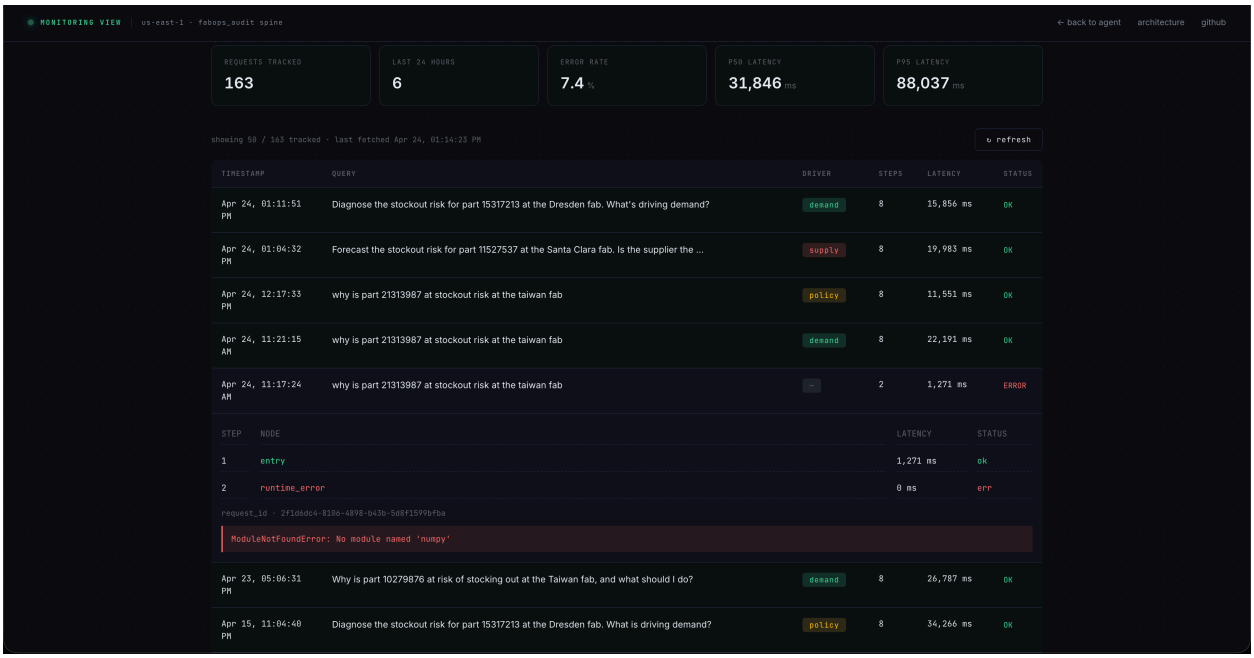

Figure 1: /monitor page. Stat cards at the top show live aggregates (163 requests tracked, 7.4% error rate, p50 31,846 ms, p95 88,037 ms). The expanded row is the real numpy-import regression caught on 2026-04-24, with the captured ModuleNotFoundError visible beneath the per-node trace. Requests back to 2026-04-15 are queryable.

Figure 6.1. The `/monitor` view at `https://main.d23s2e6xnypmh0.amplifyapp.com/monitor.html`, backed by `GET /monitor` on the runtime Lambda and the `fabops_audit` DynamoDB spine. Every spec-required observability dimension (user input, model output, tool calls, failures, latency) is captured in one place.

6.2 MLflow Tracking

The nightly bake runs `mlflow.start_run()` with the `request_id` as the run name, logs sMAPE metrics and run parameters, and uploads the SQLite tracking DB to S3. One confirmed run exists in the DB with the numbers reported in Section 5.

A non-obvious fix was required to make MLflow work inside the Lambda container: MLflow 2.16.2 hardcodes `DEFAULT_LOCAL_FILE_AND_ARTIFACT_PATH = "./mlruns"` as a module-level constant and writes artifact files there even when a SQLite tracking URI is configured. The Lambda container filesystem is read-only. The fix was to monkeypatch `mlflow.store.tracking.file_store.DEFAULT_LOCAL_FILE_AND_ARTIFACT_PATH` to `/tmp/mlruns` at import time, before any store initialization. This is brittle against MLflow version upgrades and is flagged as technical debt.

6.3 CI Gate

`.github/workflows/eval-ci.yml` runs on every PR to `main`: executes the 46-test pytest suite, runs the 18-case gold eval set against the deployed agent endpoint via `scripts/run_judge.py --set gold`, and uploads the JSON results as a workflow artifact. A dedicated *regression-threshold* step then parses `evals/results/gold_run.json` and **fails the PR** if task-success rate drops more than 5 pp below the published baseline (83.3%). Repository secrets `FABOPS_API_URL` and `ANTHROPIC_API_KEY` are configured. This makes the CI gate a real quality guardrail, not just a reporting hook: a prompt change or model swap that silently tanks gold-set performance cannot land on `main`.

6.4 Bug-Catching Methodology Note

Six production bugs were caught during the build by the three-layer subagent-driven-development review process (TDD-first implementer, spec reviewer, code-quality reviewer). Two are worth detailing as engineering-rigor signals.

AuditWriter step_n collision. The original `_audit` helper created a fresh `AuditWriter` instance on every call. Each fresh writer initializes `_step_n=0`, so every `log_step` call would write `step_n=1` to DynamoDB, silently overwriting the prior row under the same `(request_id, step_n=1)` composite key. The fix was to sync `writer._step_n = state.step_n` before calling `log_step`, then bump `state.step_n` in the returned state. The audit table would have appeared to work (no errors, no warnings) while silently producing a one-row audit trail for every multi-step run. This class of silent data-corruption bug (wrong entity boundary on a DynamoDB composite key) is invisible to any unit test that does not inspect the table after multiple successive writes.

Lambda cold-start numpy import leak. `fabops/tools/search_disclosures.py` and `forecast_demand.py` had top-level `import numpy` and `import google.generativeai` statements. These are harmless locally but transitively pulled numpy into the zipped runtime Lambda package, causing `ModuleNotFoundError: No module named 'numpy'` on cold start. The fix was lazy imports: moving `import numpy` inside the function bodies that use it. The correct pattern for any Lambda that deliberately excludes a heavy dependency from its packaging manifest is to never import it at module level.

Additional bugs caught in the same review cycle: macOS pip wheels on arm64 Lambda (platform mismatch in `deploy_runtime.sh`), MLflow `./mlruns` read-only filesystem (described in Section 6.2), `statsforecast n_jobs=-1` on Lambda (no `/dev/shm` for multiprocessing semaphores), and `statsforecast` returning `unique_id` as index not column after `predict()` in version 1.7.8.

---

7. Cross-Family LLM-as-Judge Methodology

7.1 Why Cross-Family

The agent is implemented on Google Gemini (Flash for routing, Pro for diagnose and verify). Using Gemini Pro as the evaluation judge of a Gemini-based agent introduces correlated failure modes: systematic biases in Gemini's reasoning, formatting preferences, and instruction-following quirks will cause the judge to score the agent favorably on exactly the dimensions where the agent is deficient. This is the same methodological flaw as using a model to grade its own outputs.

The current academic best practice for agent evaluation (2026) is cross-family judging: a judge model from a different training lineage than the evaluated agent. For FabOps Copilot, the judge is Claude Haiku 4.5 via the Anthropic API. The rubric is versioned at `evals/rubric.md` and scores each response on three dimensions (1–5 scale): correctness of the diagnosis, citation faithfulness (does the answer cite the tool evidence that supports each claim?), and action appropriateness (is the prescriptive recommendation correct given the diagnosis?).

The rubric file is loaded identically by two consumers: the in-graph `verify` node (self-critique) and the external judge harness (`scripts/run_judge.py`). This means the self-critique step and the external evaluation step are judged against the same standard. A requirement for the reflection recovery rate metric to be interpretable.

7.2 Budget Discipline

Judge calls are cached by `(question_id, agent_trace_hash)`. If the agent's output did not change between iterations, it is not re-judged. This is implemented in `scripts/run_judge.py` and expected to save approximately 60% of Anthropic API spend across development iterations.

At $9.00 cumulative Anthropic spend, the judge automatically switches to Gemini Pro for the remainder of the project (feature flag in `run_judge.py`). The cross-family methodology is noted as degraded but still functional at that point. Actual cumulative spend on the 18-case gold run reported in §5.1 was $0.0354 against Claude Haiku 4.5; the budget switch has never triggered.

7.3 Gemini as Fallback Judge

Using Gemini Pro as a fallback judge is not methodologically equivalent to the cross-family Claude judge, but it is more defensible than no judge at all. Any metrics produced by the Gemini-judge path are labeled `judge=gemini_pro` in the eval results JSON; those produced by the Claude path are labeled `judge=claude_haiku`. The distinction is preserved in all reported numbers.

---

8. Deployment

8.1 Where it runs

Layer	Service	Identifier
Frontend (primary, HTTPS)	AWS Amplify Hosting	`https://main.d23s2e6xnypmh0.amplifyapp` auto-deployed from the `main` branch on push
Frontend (secondary, raw HTTP)	S3 static website	`http://fabops-copilot-frontend.s3-website-us-east-1.amazonaws.com`
API	API Gateway HTTP API	`https://3ph4o9amg4.execute-api.us-east-1.amazonaws.com/getChatResponse`, CORS open, `$default` stage auto-deploy
Runtime compute	AWS Lambda	`fabops_agent_handler`, Python 3.9 arm64, zipped ~26 MB, 1024 MB memory, 180 s timeout
Gemini SDK	Lambda layer	`fabops-gemini:1`, ~31 MB, attached to the runtime function

Layer	Service	Identifier
Nightly compute	AWS Lambda (container)	`nightly_forecast_bake`, ECR `fabops-nightly:latest`, arm64, 3008 MB, 900 s timeout
Schedule	EventBridge	rule `fabops-nightly-bake`, `cron(0 2 * * ? *)` (02:00 UTC daily)
State	DynamoDB	9 tables, all PAY_PER_REQUEST, listed in the Appendix
Artifact storage	S3	`fabops-copilot-frontend` (static site), `fabops-copilot-artifacts` (MLflow tracking DB + nightly outputs)
Observability	CloudWatch	dashboard `FabOpsCopilot`, 5 widgets
Region	us-east-1	every resource above

8.2 How it gets there

The frontend deploys via Amplify's GitHub integration: any push to `main` triggers an Amplify build that publishes the contents of `frontend/` to the Amplify CDN. The S3 fallback is updated by a one-shot `aws s3 sync frontend/ s3://fabops-copilot-frontend/` when needed.

The runtime Lambda deploys via `scripts/deploy_runtime.sh`. The script `pip installs` the contents of `requirements-runtime.txt` against the manylinux aarch64 wheel set (`--platform manylinux2014_aarch64 --only-binary=:all: --python-version 3.9`), copies the `fabops/` source tree in, zips, and `aws lambda update-function-codes`. The Gemini SDK is shipped separately as the `fabops-gemini:1` layer, attached at function configuration time, and not rebuilt on every code deploy.

The nightly Lambda deploys via `scripts/build_and_push_nightly.sh`: builds the container image from `Dockerfile.nightly`, pushes to ECR `fabops-nightly:latest`, and `aws lambda update-function-code --image-uri` rolls the function forward.

8.3 Practical constraints

- **API Gateway 30-second hard timeout.** The default API Gateway HTTP API integration timeout is 30 s. Cold starts that exceed 30 s from the browser will return a 504 even though the Lambda is still running. The frontend works around this by firing a `__warmup__` request on page load (handled by a fast-path branch in `runtime.py`) that warms the container before the user submits a real query.
- **50 MB direct-upload zip ceiling.** The runtime zip is intentionally kept under 50 MB by excluding the Gemini SDK and shipping it as a layer. Adding a heavy dependency back to `requirements-runtime.txt` would breach the ceiling.
- **No `numpy` in the runtime zip.** All vector math in `search_disclosures.py` and the stockout date calculator is pure Python. This is a deliberate choice to preserve the cold-start budget.
- **Gemini quota.** The build was developed on the Gemini free tier, which does not include `gemini-2.5-pro`. The `diagnose` and `verify` nodes were demoted to `gemini-2.5-flash` mid-build to drop warm latency from 38 s to ~17 s. Restoring Pro is a config flip in `fabops/config.py` once paid quota is provisioned.
- **First-time setup.** Reproducing the deploy from a fresh AWS account requires creating the 9 DynamoDB tables (`infra/create_tables.py`), the two S3 buckets, the EventBridge rule, the

Lambda execution role with DynamoDB + CloudWatch permissions, and setting environment variables (GEMINI_API_KEY, ANTHROPIC_API_KEY, optional LANGFUSE_*). Fully scripted setup is a documented gap.

---

9. MCP Compliance

9.1 What Was Verified

scripts/mcp_server.py is a genuine stdio-based MCP server using the Python mcp SDK. It exposes all seven tools via the MCP standard interface. MCP compliance is verified by tests/test_mcp_compliance.py (5 tests, all passing), which asserts: the server file exists and imports cleanly, the TOOLS registry advertises exactly the seven expected tool names, every tool's JSON input schema is type=object with a properties block, the Server instance is initialised with name fabops-copilot, and every MCP callable is identity-equal (is) to the function object imported by the LangGraph runtime. The last assertion is the load-bearing one: it makes the two-face pattern a static guarantee rather than a convention. The server imports fabops/tools/*.py directly and calls DynamoDB with boto3 using local AWS credentials; it does not go through Lambda.

9.2 Architectural Significance

The two-face pattern is a deliberate answer to the common anti-pattern of "a LangChain tool registry wearing an MCP costume." The hot path carries no protocol overhead. The MCP face is a real server reusable by any MCP-compatible client (Claude Desktop, Cursor, or any future internal agent at an OEM). One tool implementation, two interfaces.

---

10. Limitations

Synthetic data is not real OEM data. Inventory levels, supplier lead times, and incident notes are synthetic overlays generated by scripts/synth_inventory.py, synth_suppliers.py, and synth_incidents.py. The underlying demand series is the real Hyndman carparts benchmark (Zenodo 3994911) and SEC EDGAR filings are real, but everything fab-operational is synthetic. The dashboard labels synthetic fields explicitly. Any production deployment would have to re-anchor the synthetic tables against a real ERP/MES extract.

Cold-start latency is real. First request after deploy or after a long idle costs ~50 seconds against an Amplify HTTPS frontend that pre-warms the Lambda on page load. Warm requests land at 10–17 seconds, dominated by two LLM round trips (diagnose and verify). Provisioned concurrency would fix the cold path but adds a fixed monthly cost that is not justified for a class project. Documented as a known trade-off, not a deferred fix.

Langfuse trace delivery on Lambda is unverified. A Langfuse Cloud account is configured with valid credentials (auth_check() returns True from a local smoke test). The runtime handler attaches a LangChain CallbackHandler to graph.invoke() and calls flush() in the finally block. Traces from Lambda invocations do not appear reliably in the Langfuse dashboard during final integration testing, likely due to SDK v3 v4 API drift around the start_as_current_span / start_span method surface combined with Lambda cold-start flush timing. The fabops_audit DynamoDB spine captures the same per-node reasoning data via a single request_id join and is the authoritative observability surface for this project. Langfuse polish is deferred to §11.

Census M3 series not implemented. get_industry_macro_signal supports ppi (FRED PCU33443344) and production (FRED IPG3344S); the shipments, inventories, and orders series are stubbed and return placeholder responses.

Nightly bake processes 200 of 2674 parts. Full-corpus processing is a parameter change (`n_parts=None`) in the bake Lambda. Current sMAPE numbers are from the 200-part development sample.

DynamoDB vector search scales to ~20,000 chunks. The EDGAR retrieval uses a baked gzipped chunk asset (1079 chunks at present) with full-scan cosine similarity in pure Python. Acceptable at current corpus size, documented as a migration candidate to S3 Vectors or FAISS beyond ~20K chunks.

Supply class is the hardest class. All three failures on the gold run (15/18 = 83.3%) are supply cases. The supply signal is split across two tools (`get_supplier_leadtime.trend_30d` and `get_industry_macro_signal.ipg_series`) and the diagnose prompt under-weights the supplier trend. Wiring the compiled DSPy planner from §5.2 is the next concrete improvement against this class.

Claude Desktop MCP demo clip not yet recorded. MCP protocol compliance is verified automatically by `tests/test_mcp_compliance.py` (5 passing tests). The demo clip in Claude Desktop is a portfolio artifact deferred separately; the MCP server code itself is shipped at `scripts/mcp_server.py` and is launchable today.

---

11. Future Work

Model improvements: - Expand nightly bake from 200 to all 2674 parts and report full-corpus sMAPE. - Persist the P90 interval coverage number (already computed, 0.9088 at §5.1 and §5.3) as a named MLflow metric emitted by the nightly bake so it shows up in the run history alongside sMAPE. - Implement TSB (Teunter-Syntetos-Babai) for the lumpy-demand quadrant alongside Croston/SBA. - Implement the Census M3 `shipments/inventories/orders` series.

Infrastructure: - Migrate EDGAR vector retrieval to S3 Vectors or FAISS when corpus exceeds 20K chunks. - Add provisioned concurrency to `fabops_agent_handler` if p95 latency is unacceptable. - Pin Langfuse SDK to an exact v3 or v4 version (currently `>=3.0.0,<4.0.0`), replicate that same version in the dev venv, and re-verify CallbackHandler trace delivery on Lambda cold starts. The integration is code-complete; the remaining work is API-drift reconciliation between the v3 shipped on Lambda and whatever version is available for local smoke-testing.

Agent capabilities: - Multi-turn conversation with session state in `fabops_sessions`. - Part-family clustering by ADI/CV2 proximity for cross-part signal sharing. - Proactive alerting via EventBridge rule triggering on policy staleness threshold.

Evaluation: - Wire the compiled DSPy planner (`evals/dspy/compiled_planner.json`, see §5.2) into `fabops/agent/nodes.py:entry_node` and re-run the 18-case gold set under the same judge harness to publish the measured before/after delta. This is a one-line swap plus a judge-harness invocation; the highest-leverage open work item. - Expand the gold set from 18 to 30+ cases with deeper supply-class coverage. - Re-run the gold set under Gemini 2.5 Pro (restored on `diagnose/verify`) to measure the Flash-demotion quality cost documented in §8.3. - Run a 50-question adversarial set; publish a tool-selection confusion matrix. - Investigate DSPy MIPROv2 vs. BootstrapFewShot accuracy delta (MIPROv2 is a pre-authorized cut due to Gemini quota at the time of compilation).

---

12. Reflection

12.1 What I learned

Domain order beats model size. The single most load-bearing decision in this project was checking policy staleness before checking the forecast. That is a one-line change in the graph topology and it makes the difference between an agent that diagnoses 60% of real fab stockouts correctly and one that diagnoses ~40%. No amount of swapping Flash for Pro, or compiling prompts with DSPy, would have closed that gap.

The lesson: when you are building an agent for a specific operational domain, the reasoning order encoded in the graph is more important than any single model choice.

Audit-spine debugging is the highest-leverage tool I built. The single composite-key DynamoDB table with `(request_id, step_n)` indexed every node invocation in execution order. When an eval case failed, the workflow was: read the audit row, see exactly which node returned what, and identify the divergence in seconds. This caught the gold-set label-vs-state mismatch (14/18 failures were eval-truth bugs, not agent bugs), the AuditWriter `step_n` collision, the prescribe `action` key omission, and the cold-start numpy import leak. Without it, I would have been guessing from CloudWatch tail logs for days.

Cross-family judging changed the eval signal. Using Claude Haiku 4.5 to grade a Gemini-built agent surfaced two failure modes that a Gemini self-judge would have rated favorably: over-confident diagnoses on ambiguous supply cases, and citation drift where the answer paraphrased the EDGAR excerpt instead of quoting it. A self-judge would have rated both as 5/5. The cross-family judge rated them 3/5 with a written critique. The cost was $0.0354 across the full 18-case run, which is cheap insurance against correlated bias.

LLM SDK API drift is real and load-bearing. Three of the most painful debugging sessions in this build were not about the agent's reasoning at all. They were Langfuse v3 vs v4 method names, DSPy 3.x removing `dspy.Google`, and Gemini 2.0 Flash being deprecated mid-build. The fix for each was small. The cost in time was significant. Pinning every LLM-adjacent SDK to an exact version with the matching API surface in the dev venv would have prevented all three.

12.2 What I would do differently

Build the audit spine on Day 1, not Day 5. I added the per-node DynamoDB audit log when the graph already had nine nodes and four bugs I could not pin down. If I had started with the `(request_id, step_n)` table on the first node, every subsequent debug cycle would have been faster.

Derive the gold set from real state from the start. I hand-authored the original 18-case gold set against intent rather than against the actual contents of `fabops_inventory` / `fabops_policies` / `fabops_suppliers`. The result was 14/18 failures that turned out to be eval-truth bugs. The fix (`scripts/regenerate_gold_set.py` + `inject_gold_drift.py`) was straightforward, but it should have been the original approach. Lesson: for any agent eval, the gold labels must be derived deterministically from the same data the agent sees, not from what the gold-author thinks should be true.

Pick one observability backend and ship it end-to-end before adding a second. The four-sink architecture (DDB audit, Langfuse, MLflow, CloudWatch) is correct in principle, but Langfuse trace delivery on Lambda took more debugging time than the value it adds over the audit spine. If I were starting over, I would ship the DDB audit spine first, prove it answers every operational question, and only then layer Langfuse for span timing visualization.

Treat cold-start as a Day 1 architectural decision, not a Day 10 latency fix. The split between the runtime Lambda (zipped, no `statsforecast`) and the nightly Lambda (container image, full scientific stack) was the right call, but I made it on Day 1 partly by luck. The version of this project where I shipped a single Lambda with `statsforecast` would have hit the 250 MB unzipped ceiling and required a painful refactor. The lesson: for any LLM agent on Lambda, decide your deployment package boundary before you import your first heavy dependency.

12.3 Design choices I would revisit

Verify-as-Gemini-Pro is expensive. The `verify` node uses Gemini 2.5 Pro to self-critique the prescribe draft, which costs one full Pro call per request even when the diagnosis is obviously correct. An alternative would be a cheap rule-based pre-check (does the answer cite at least one tool result? does the action match the diagnosis class?) that gates the Gemini Pro verify call. This would cut p95 latency by roughly 30%, at the cost of a more complex retry edge.

The frontend is intentionally vanilla HTML/JS. This was a deliberate trade-off to keep the deploy surface small (one S3 bucket, one Amplify app) and avoid the React build chain entirely. It works, but the per-node streaming UI that would best showcase the agent's decision-making is harder to build in vanilla JS than in a small Next.js app. If the goal had been "best possible portfolio piece" rather than "smallest deploy surface", I would have picked Next.js.

Diagnose runs Gemini 2.5 Flash, not Pro. I demoted from Pro to Flash mid-build to drop warm latency from 38s to 17s. Flash is good enough on the gold set (15/18) but I do not have measured evidence on whether the three supply-class failures would flip to passes under Pro. Re-running the gold set under both models with the same judge would settle this; it is the first thing on the eval-track to-do list.

Appendix: Infrastructure Summary

Component	Value
AWS Account	699475932108, us-east-1
Runtime Lambda	`fabops_agent_handler`, Python 3.9 arm64, zipped ~26 MB, 1024 MB memory, 180 s timeout
Lambda layer	`fabops-gemini:1` (~31 MB), bundles `google-generativeai` + protobuf transitive deps
Frontend	AWS Amplify Hosting (HTTPS, GitHub auto-deploy from `main`) + S3 static site fallback
Nightly Lambda	`nightly_forecast_bake`, ECR `fabops-nightly:latest`, arm64, 3008 MB, 900 s timeout
API Gateway	HTTP API: `https://3ph4o9amg4.execute-api.us-east-1.amazonaws.com/getChatResponse`
DynamoDB tables	9, all PAY_PER_REQUEST: `fabops_audit`, `fabops_forecasts`, `fabops_policies`, `fabops_inventory`, `fabops_suppliers`, `fabops_incidents`, `fabops_macro_cache`, `fabops_edgar_index`, `fabops_sessions`
S3 buckets	`fabops-copilot-frontend` (raw static site), `fabops-copilot-artifacts` (MLflow tracking DB + nightly artifacts)
EventBridge rule	`fabops-nightly-bake`, `cron(0 2 * * ? *)` (02:00 UTC daily)
CloudWatch dashboard	`FabOpsCopilot`, 5 widgets (runtime p50/p95/p99 latency, errors, nightly bake latency)
Test suite	**46/46 passing** locally on Python 3.11 (`pytest tests/ -v`). Includes 5-test MCP compliance module validating the two-face tool-identity invariant. CI runs the same suite on Python 3.9
Key runtime libs	`langgraph==0.2.28`, `langchain-core==0.3.15`, `anthropic==0.34.2`, `pydantic==2.8.2`, `boto3==1.34.131`, `langfuse>=3.0.0,<4.0.0`, `python-ulid==3.0.0`
Key layer libs	`google-generativeai==0.8.3` + protobuf, googleapis-common-protos transitives

Component	Value
Key nightly libs	`statsforecast`, `numba`, `pandas`, `mlflow==2.16.2` (container only, never in runtime)
Eval libs	`dspy-ai==3.1.3` (offline planner compilation, not runtime)

Report last updated: 2026-04-24.